Assignment #4 — Facade Pattern: Online Order Processing

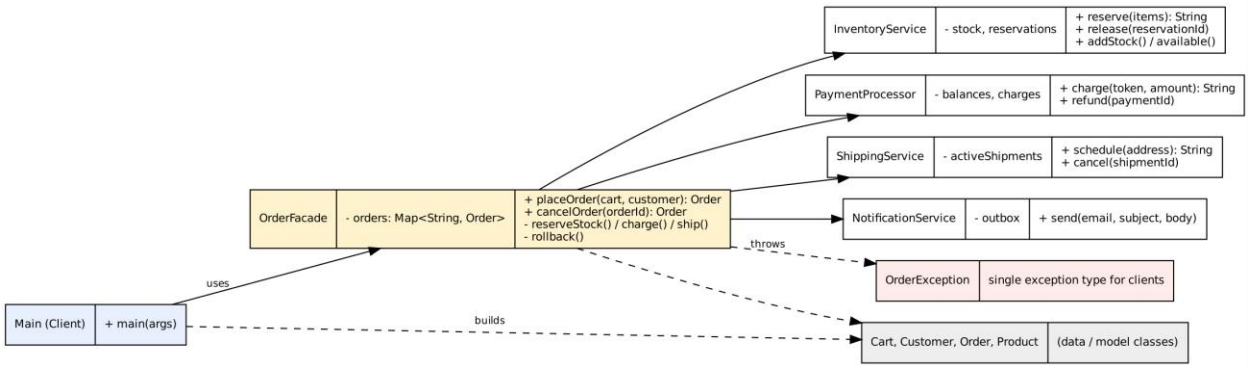
Student: Baimagambet Gauhar Group: SE-2530 Option: B

1. Introduction

Placing an online order is not one action but a coordinated sequence across four services: InventoryService (reserve stock), PaymentProcessor (charge the customer), ShippingService (book delivery) and NotificationService (send e-mail). The order matters, each step returns an id the next one or a later cancellation needs, and if a step fails the earlier ones must be undone (for example, stock reserved before a declined payment must be released).

Without a Facade every client (web controller, mobile API, admin tool) would repeat this choreography and could get it wrong, for instance forgetting to release stock. OrderFacade solves this with two methods: placeOrder(cart, customer) and cancelOrder(orderId). The Facade pattern fits because it provides a simple, unified interface to a complex subsystem without changing the subsystem classes.

2. UML class diagram



The Client depends only on OrderFacade (plus plain data classes). The Facade depends on the four services. Arrows point one way: the services know nothing about the Facade.

3. Clean Code principles

3.1 Meaningful names

placeOrder, cancelOrder, reserve, release, charge and refund state intent, so the API reads like the business process and needs no comments.

```
Before: facade.doStuff(a, b, 3);
After:  Order order = shop.placeOrder(cart, customer);
        shop.cancelOrder(order.id());
```

3.2 Single Responsibility — the Facade coordinates, it does not reimplement

Stock arithmetic lives in InventoryService, balance checks in PaymentProcessor, and so on. The Facade only sequences calls and compensates on failure.

```
String reservationId = reserveStock(cart);   // delegates to InventoryService
paymentId = charge(cart, customer);          // delegates to PaymentProcessor
String shipmentId = ship(customer);          // delegates to ShippingService
```

3.3 No leaking of subsystem details

The public API exposes only Cart, Customer, Order and one exception type. Subsystem exceptions (PaymentDeclinedException etc.) are translated, so clients never import subsystem packages.

```
Before: catch (OutOfStockException | PaymentDeclinedException | ShippingException e)
After:  catch (OrderException e)   // thrown by the facade

private String charge(...) throws OrderException {
    try { return payments.charge(token, total); }
    catch (PaymentDeclinedException e) {
        throw new OrderException("Payment declined: " + e.getMessage(), e);
    }
}
```

3.4 Small, focused subsystem classes

Each service has one reason to change and a handful of methods; the Facade methods are short and private helpers (reserveStock, charge, ship, rollback) keep placeOrder readable.

```
InventoryService: addStock, available, reserve, release
PaymentProcessor: registerAccount, balanceOf, charge, refund
ShippingService:  schedule, cancel
NotificationService: send
```

3.5 Proper error handling and rollback

A failure after a successful step triggers compensation in reverse order, keeping the system consistent. The original cause is preserved via exception chaining.

```
try {
    paymentId = charge(cart, customer);
    String shipmentId = ship(customer);
```

```
    ...
} catch (OrderException e) {
    rollback(reservationId, paymentId);   // refund + release stock
    throw e;
}
```

3.6 Dependency injection / loose coupling

Services are passed through the Facade constructor rather than created inside it, which makes the Facade testable and keeps wiring in the client (composition root).

```
public OrderFacade(InventoryService inventory, PaymentProcessor payments,
                   ShippingService shipping, NotificationService notifications)
```

Demonstration results

Running Main shows: (1) a successful order lowers stock and balance; (2) a declined payment leaves stock and balances unchanged; (3) a shipping failure refunds the payment and releases stock; (4) insufficient stock is rejected; (5) cancelling restores stock and refunds the customer.

4. Conclusion

What the Facade hides: the order of operations, the ids passed between services, translation of four different exception types into one, and the rollback logic.

What remains directly accessible: the subsystem classes have no back-references and are still public, so an advanced client (for example an admin tool restocking inventory, or a refund-only service) can use InventoryService or PaymentProcessor directly. The Facade is a convenience, not a gate. The demo client uses subsystem classes only to wire the Facade and seed test data.

5. GitHub repository

https://github.com/gauharb/sdp-asn4